

THỰC HÀNH THIẾT KẾ HỆ THỐNG SOC - LỚP **CE433.Q21.1**

BÀI THỰC HÀNH 2

Giảng viên hướng dẫn	Trương Văn Cường		ĐIỂM
Sinh viên thực hiện	Vũ Đại Dương	23520359	

1. Buổi 3: Giao tiếp IO tự thiết kế

Bài 1. Đổi code ở file source.c để truy xuất 2 mô đun Switches và Red_leds từ Nios II thông qua con trỏ.

Bài 2. Thực hiện hệ thống SoC có nhiệm vụ giải mã led 7 đoạn tương ứng với dữ liệu từ SW[3:0] như code đã chuẩn bị ở bài chuẩn bị ở nhà.

2. Buổi 4: Thiết kế module chức năng và khảo sát bằng signal tab

Bài 1. Thực hiện lại hệ thống như bài hướng dẫn thực hành, chỉnh sửa code C để truy xuất mô đun Compute từ Nios II thông qua con trỏ.

Bài 2. Xây dựng hệ thống SoC như bài hướng dẫn thực hành, nhưng các giá trị a, b, x được nhập từ SW và kết quả xuất ra LEDG như sau:

- a= SW[3:0].
- b= SW[7:4].
- c=SW[11:8].
- LEDG[7:0]=y.
- Bắt dữ liệu để hiển thị trên Signal Tab.

3. Báo cáo thực hành

a. Buổi 3: Giao tiếp IO tự thiết kế

i. Bài 1

- Thiết kế hệ thống trên Qsys:

Use	Connections	Name	Description	Export	Clock	Base	End	IRQ	Opcode Name
☒		☐ clock	Clock Source						
		clk_in	Clock Input	clk					
		clk_in_reset	Reset Input	reset					
		clk	Clock Output	Double-click to export	clock				
		clk_reset	Reset Output	Double-click to export					
☒		☐ nios2_qsys_0	Nios II Processor						
		clk	Clock Input	Double-click to export	clock				
		reset_n	Reset Input	Double-click to export	[clk]				
		data_master	Avalon Memory Mapped Master	Double-click to export	[clk]			IRQ 0	
		instruction_master	Avalon Memory Mapped Master	Double-click to export	[clk]				
		jtag_debug_module_re...	Reset Output	Double-click to export	[clk]				
		jtag_debug_module	Avalon Memory Mapped Slave	Double-click to export	[clk]	# 0x0001_0800	0x0001_0fff		
		custom_instruction_m...	Custom Instruction Master	Double-click to export					
☒		☐ onchip_memory2_0	On-Chip Memory (RAM or ROM)						
		clk1	Clock Input	Double-click to export	clock				
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk1]	# 0x0000_8000	0x0000_ffff		
		reset1	Reset Input	Double-click to export	[clk1]				
☒		☐ jtag_uart_0	JTAG UART						
		clk	Clock Input	Double-click to export	clock				
		reset	Reset Input	Double-click to export	[clk]	# 0x0001_1008	0x0001_100f		
		avalon_jtag_slave	Avalon Memory Mapped Slave	Double-click to export					
☒		☐ red_leds_0	red_leds						
		avalon_slave_0	Avalon Memory Mapped Slave	Double-click to export	[clock_sink]	# 0x0001_1010	0x0001_1013		
		clock_sink	Clock Input	Double-click to export	clock				
		reset_sink	Reset Input	Double-click to export	[clock_sink]				
		conduit_end	Conduit	red_leds_0_conduit_end					
☒		☐ switches_0	switches						
		avalon_slave_0	Avalon Memory Mapped Slave	Double-click to export	[clock_sink]	# 0x0001_1014	0x0001_1017		
		reset_sink	Reset Input	Double-click to export	[clock_sink]				
		conduit_end	Conduit	switches_0_conduit_end					
		clock_sink	Clock Input	Double-click to export	clock				

Hình 1 Hệ thống trên Qsys

- Tích hợp hệ thống:

```

module Bai3(
    input                CLOCK_50,
    input    [0:0]      KEY,
    input    [15:0]     SW,
    output   [15:0]     LEDR
);
system Nios_system(
    .clk_clk                (CLOCK_50),
    .reset_reset_n         (KEY[0]),
    .red_leds_0_conduit_end_export (LEDR),
    .switches_0_conduit_end_export (SW)
);

endmodule

```

Hình 2 Code verilog Bai3.v

- Code truy xuất 2 mô đun Switches và Red_leds từ Nios II thông qua con trỏ trong Eclipse:

```

#include <stdio.h>
#include <system.h>

int main() {
    volatile short *switches_ptr = (volatile short *) SWITCHES_0_BASE;
    volatile short *red_leds_ptr = (volatile short *) RED_LEDS_0_BASE;

    short temp;

    while (1) {
        temp = *switches_ptr;
        *red_leds_ptr = temp;
    }

    return 0;
}

```

Hình 3 Code C theo yêu cầu

- Build Project:

```

CDT Build Console [Bai3]
the public mailing list cygwin@cygwin.com
2 [main] sh 16836 find_fast_cwd: WARNING: Couldn't compute FAST_CWD pointer. Please report this problem to
the public mailing list cygwin@cygwin.com
[BSP build complete]
1 [main] true 11628 find_fast_cwd: WARNING: Couldn't compute FAST_CWD pointer. Please report this problem to
the public mailing list cygwin@cygwin.com
1 [main] sh 16068 find_fast_cwd: WARNING: Couldn't compute FAST_CWD pointer. Please report this problem to
the public mailing list cygwin@cygwin.com
[Bai3 build complete]

**** Build Finished ****

```

Hình 4 Kết quả Build Project

- Nạp KIT thực hành (*)

ii. Bài 2

- Thiết kế 2 file switches.v cho switch và decoder_7_seg cho LED 7 đoạn:

```

module switches (
    input          iClk,
    input          iReset_n,
    input          iChip_select_n,
    input          iRead_n,
    input [31:0]   iSwitches_data,
    output reg [31:0] oRead_data
);
    always @(posedge iClk or negedge iReset_n) begin
        if (~iReset_n)
            oRead_data <= 32'd0;
        else if (~iChip_select_n && ~iRead_n)
            oRead_data <= iSwitches_data;
        end
    endmodule

```

Hình 5 Code verilog switches.v

```

module decoder_7_seg (
    input          iClk,
    input          iReset_n,
    input          iChip_select_n,
    input          iWrite_n,
    input [31:0]   iWrite_data,
    output reg [6:0] oHEX_physical
);
    always @(posedge iClk or negedge iReset_n) begin
        if (~iReset_n)
            oHEX_physical <= 7'b1111111;
        else if (~iChip_select_n && ~iWrite_n) begin
            case (iWrite_data[3:0])
                4'h0: oHEX_physical <= 7'b1000000;
                4'h1: oHEX_physical <= 7'b1111001;
                4'h2: oHEX_physical <= 7'b0100100;
                4'h3: oHEX_physical <= 7'b0110000;
                4'h4: oHEX_physical <= 7'b0011001;
                4'h5: oHEX_physical <= 7'b0010010;
                4'h6: oHEX_physical <= 7'b0000010;
                4'h7: oHEX_physical <= 7'b1111000;
                4'h8: oHEX_physical <= 7'b0000000;
                4'h9: oHEX_physical <= 7'b0010000;
                4'hA: oHEX_physical <= 7'b0001000;
                4'hB: oHEX_physical <= 7'b0000011;
                4'hC: oHEX_physical <= 7'b1000110;
                4'hD: oHEX_physical <= 7'b0100001;
                4'hE: oHEX_physical <= 7'b0000110;
                4'hF: oHEX_physical <= 7'b0001110;
                default: oHEX_physical <= 7'b1111111;
            endcase
        end
    end
endmodule

```

Hình 6 Code verilog decoder_7_seg.v

- Thiết kế hệ thống trên Qsys:

Name	Interface	Signal Type	Width	Direction
iChip_select_n	avalon_slave_0	chipselect_n	1	input
iRead_n	avalon_slave_0	read_n	1	input
oRead_data	avalon_slave_0	readdata	32	output
iClk	clock_sink	clk	1	input
iReset_n	reset_sink	reset_n	1	input
iSwitches_data	conduit_end	export	32	input

Hình 7 Cấu hình tín hiệu cho switches

Name	Interface	Signal Type	Width	Direction
iChip_select_n	avalon_slave_0	chipselect_n	1	input
iWrite_n	avalon_slave_0	write_n	1	input
iWrite_data	avalon_slave_0	writedata	32	input
iClk	clock_sink	clk	1	input
iReset_n	reset_sink	reset_n	1	input
oHEX_physical	conduit_end	export	7	output

Hình 8 Cấu hình tín hiệu cho decoder_7_seg

Use	Connections	Name	Description	Export	Clock	Base	End	IRQ	Opcode Name
☒		☐ clock	Clock Source						
		clk_in	Clock Input	clk					
		clk_in_reset	Reset Input	reset					
		clk	Clock Output	Double-click to export	clock				
		clk_reset	Reset Output	Double-click to export					
☒		☐ nios2_qsys_0	Nios II Processor						
		clk	Clock Input	Double-click to export	clock				
		reset_n	Reset Input	Double-click to export	[clk]				
		data_master	Avalon Memory Mapped Master	Double-click to export	[clk]				
		instruction_master	Avalon Memory Mapped Master	Double-click to export	[clk]				
		jtag_debug_module_re	Reset Output	Double-click to export	[clk]				
		jtag_debug_module	Avalon Memory Mapped Slave	Double-click to export	[clk]	# 0x0001_0800	0x0001_0fff		
		custom_instruction_m	Custom Instruction Master	Double-click to export					
☒		☐ onchip_memory2_0	On-Chip Memory (RAM or ROM)						
		clk1	Clock Input	Double-click to export	clock				
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk1]	# 0x0000_8000	0x0000_ffff		
		reset1	Reset Input	Double-click to export					
☒		☐ jtag_uart_0	JTAG UART						
		clk	Clock Input	Double-click to export	clock				
		reset	Reset Input	Double-click to export	[clk]				
		avalon_jtag_slave	Avalon Memory Mapped Slave	Double-click to export	[clk]	# 0x0001_1008	0x0001_100f		
☒		☐ switches_0	switches						
		avalon_slave_0	Avalon Memory Mapped Slave	Double-click to export	[clock_sink]	# 0x0001_1010	0x0001_1013		
		clock_sink	Clock Input	Double-click to export	clock				
		reset_sink	Reset Input	Double-click to export	[clock_sink]				
		conduit_end	Conduit	switches_0_conduit_end					
☒		☐ decoder_7_seg_0	decoder_7_seg						
		avalon_slave_0	Avalon Memory Mapped Slave	Double-click to export	[clock_sink]	# 0x0001_1014	0x0001_1017		
		clock_sink	Clock Input	Double-click to export	clock				
		reset_sink	Reset Input	Double-click to export	[clock_sink]				
		conduit_end	Conduit	decoder_7_seg_0_condu...					

Hình 9 Hệ thống trên Qsys

- Tích hợp phần cứng:

```

module Bai32 (
    input          CLOCK_50,
    input [0:0]    KEY,
    input [3:0]    SW,
    output [6:0]   HEX0
);

    system Nios_system (
        .clk_clk          (CLOCK_50),
        .reset_reset_n    (KEY[0]),
        .switches_0_conduit_end_export (SW),
        .decoder_7_seg_0_conduit_end_export (HEX0)
    );

endmodule

```

Hình 10 Code verilog Bai32.v

- Code giải mã led 7 đoạn tương ứng với dữ liệu từ SW[3:0]:

```

#include <stdio.h>
#include <system.h>
#include <unistd.h>

int main() {
    volatile int *sw_ptr = (volatile int *) SWITCHES_0_BASE;
    volatile int *hex_ptr = (volatile int *) DECODER_7_SEG_0_BASE;

    int current_val;
    int i;

    while (1) {
        current_val = *sw_ptr;
        *hex_ptr = current_val;

        usleep(100000);
    }

    return 0;
}

```

Hình 11 Code C theo yêu cầu

- Build Project:

```

CDT Build Console [Bai32]
the public mailing list cygwin@cygwin.com
make --no-print-directory -C ../Bai32_bsp/
  2 [main] pwd 18416 find_fast_cwd: WARNING: Couldn't compute FAST_CWD pointer. Please report this problem to
the public mailing list cygwin@cygwin.com
  2 [main] sh 17496 find_fast_cwd: WARNING: Couldn't compute FAST_CWD pointer. Please report this problem to
the public mailing list cygwin@cygwin.com
  2 [main] sh 13020 find_fast_cwd: WARNING: Couldn't compute FAST_CWD pointer. Please report this problem to
the public mailing list cygwin@cygwin.com
[BSP build complete]
  2 [main] true 5760 find_fast_cwd: WARNING: Couldn't compute FAST_CWD pointer. Please report this problem to
the public mailing list cygwin@cygwin.com
  2 [main] sh 10120 find_fast_cwd: WARNING: Couldn't compute FAST_CWD pointer. Please report this problem to
the public mailing list cygwin@cygwin.com
[Bai32 build complete]

**** Build Finished ****

```

Hình 12 Kết quả Build Project

- Nạp KIT thực hành (*)

b. Buổi 4: Thiết kế Mô đun chức năng và khảo sát bằng Signal Tab

i. Bài 1

- Thiết kế hệ thống trên Qsys:

- Xây dựng phần mềm để truy xuất mô đun Compute từ Nios II thông qua con trỏ:

```
#include <stdio.h>
#include <system.h>

typedef struct {
    volatile int a;
    volatile int b;
    volatile int x;
    volatile int result;
} Compute_Regs;

int main() {
    Compute_Regs *compute_ptr = (Compute_Regs *) COMPUTE_0_BASE;

    int val_a, val_b, val_x, val_y;

    val_a = 2 & 0xf;
    val_b = 3 & 0xf;
    val_x = 5 & 0xf;

    compute_ptr->a = val_a;
    compute_ptr->b = val_b;
    compute_ptr->x = val_x;

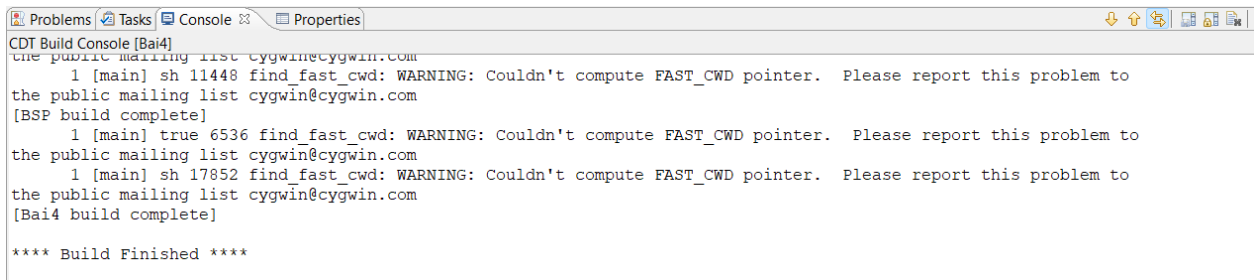
    val_y = compute_ptr->result;

    printf("y = %d*x + %d = %d, x = %d\n", val_a, val_b, val_y, val_x);

    while(1);
}
```

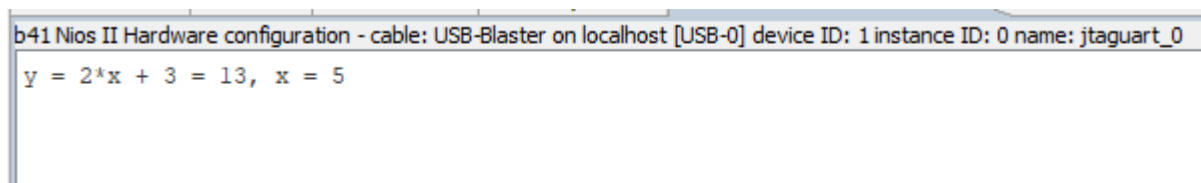
Hình 16 Code C theo yêu cầu

- Build Project:



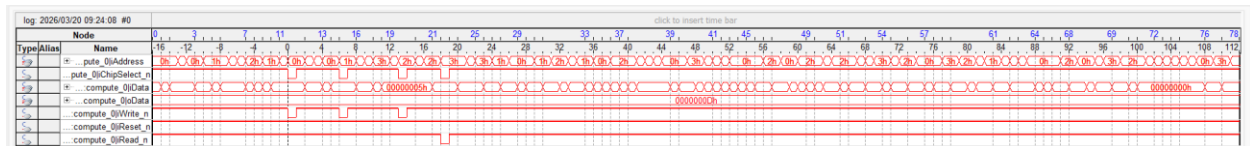
Hình 17 Kết quả Build Project

- Kết quả trên Console:



Hình 18 Kết quả trên Console

- Kết quả trên Signal Tab:



Hình 19 Kết quả trên SignalTap

ii. Bài 2:

- Module switches.v

```
module switches (
    input                iClk,
    input                iReset_n,
    input                iChip_select_n,
    input                iRead_n,
    input [31:0]         iSwitches_data,
    output reg [31:0]    oRead_data
);
    always @(posedge iClk or negedge iReset_n) begin
        if (~iReset_n)
            oRead_data <= 32'd0;
        else if (~iChip_select_n && ~iRead_n)
            oRead_data <= iSwitches_data;
        end
    end
endmodule
```

Hình 20 Code verilog switches.v

- Module red_leds.v

```

module red_leds
(
    input iClk,
    input iReset_n,
    input iChip_select_n,
    input iWrite_n,
    input [31:0] iRed_leds_data,
    output reg [31:0] oRed_leds
);
always@(posedge iClk, negedge iReset_n)
begin
    if(~iReset_n)
    begin
        oRed_leds<=32'd0;
    end
    else
    begin
        if(~iChip_select_n&~iWrite_n)
        begin
            oRed_leds <= iRed_leds_data;
        end
    end
end
endmodule

```

Hình 21 Code verilog red_leds.v

- Module compute.v

```

module Compute (
    input      iClk,
    input      iReset_n,
    input      iChipSelect_n,
    input      iWrite_n,
    input      iRead_n,
    input  [1:0] iAddress,
    input  [31:0] iData,
    output reg [31:0] oData
);

    reg [31:0] a, b, x;

    always @(posedge iClk or negedge iReset_n) begin

```

```

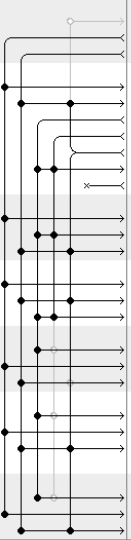
if (~iReset_n) begin
    oData <= 32'd0;
    a    <= 32'd0;
    b    <= 32'd0;
    x    <= 32'd0;
end
else begin
    if (~iChipSelect_n && ~iWrite_n) begin
        case (iAddress)
            2'd0: a <= {28'd0, iData[3:0]};
            2'd1: b <= {28'd0, iData[3:0]};
            2'd2: x <= {28'd0, iData[3:0]};
            default: ;
        endcase
    end

    if (~iChipSelect_n && ~iRead_n) begin
        case (iAddress)
            2'd0: oData <= a;
            2'd1: oData <= b;
            2'd2: oData <= x;
            2'd3: oData <= (a * x) + b;
            default: oData <= 32'd0;
        endcase
    end
end
end
endmodule

```

Bảng 1 Code verilog compute.v

- Thiết kế hệ thống Qsys:

Use	Connections	Name	Description	Export	Clock	Base	End	IRQ	Opcode Name
☒		☐ clock	Clock Source	clk					
		clk_in	Clock Input	reset					
		clk_in_reset	Reset Input						
		clk	Clock Output	Double-click to export	clock				
		clk_reset	Reset Output	Double-click to export					
☒		☐ nios2_qsys_0	Nios II Processor						
		clk	Clock Input	Double-click to export	clock				
		reset_n	Reset Input	Double-click to export	[clk]				
		data_master	Avalon Memory Mapped Master	Double-click to export	[clk]				
		instruction_master	Avalon Memory Mapped Master	Double-click to export	[clk]				
		jtag_debug_module_re...	Reset Output	Double-click to export	[clk]				
		jtag_debug_module	Avalon Memory Mapped Slave	Double-click to export	[clk]	# 0x0001_0800	0x0001_0fff		
		custom_instruction_m...	Custom Instruction Master	Double-click to export					
☒		☐ onchip_memory2_0	On-Chip Memory (RAM or ROM)						
		clk1	Clock Input	Double-click to export	clock				
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk1]	# 0x0000_8000	0x0000_ffff		
		reset1	Reset Input	Double-click to export					
☒		☐ jtag_uart_0	JTAG UART						
		clk	Clock Input	Double-click to export	clock				
		reset	Reset Input	Double-click to export	[clk]				
		avalon_jtag_slave	Avalon Memory Mapped Slave	Double-click to export	[clk]	# 0x0001_1018	0x0001_101f		
☒		☐ compute_0	compute						
		avalon_slave_0	Avalon Memory Mapped Slave	Double-click to export	[clock_sink]	# 0x0001_1000	0x0001_100f		
		clock_sink	Clock Input	Double-click to export	clock				
		reset_sink	Reset Input	Double-click to export	[clock_sink]				
☒		☐ switches_0	switches						
		avalon_slave_0	Avalon Memory Mapped Slave	Double-click to export	[clock_sink]	# 0x0001_1024	0x0001_1027		
		clock_sink	Clock Input	Double-click to export	clock				
		reset_sink	Reset Input	Double-click to export	[clock_sink]				
		conduit_end	Conduit	switches_0_conduit_end					
☒		☐ ledg_0	ledg						
		avalon_slave_0	Avalon Memory Mapped Slave	Double-click to export	[clock_sink]	# 0x0001_1020	0x0001_1023		
		clock_sink	Clock Input	Double-click to export	clock				
		reset_sink	Reset Input	Double-click to export	[clock_sink]				
		conduit_end	Conduit	ledg_0_conduit_end					

Hình 22 Hệ thống trên Qsys

- Module top:

```

module Bai42 (
    input                CLOCK_50,
    input                [0:0] KEY,
    input                [11:0] SW,
    output               [7:0] LEDG
);

    system u0 (
        .clk_clk          (CLOCK_50),
        .reset_reset_n    (KEY[0]),

        .switches_0_conduit_end_export    (SW),

        .ledg_0_conduit_end_export        (LEDG)
    );

endmodule

```

Hình 23 Code verilog Bai42.v

- Cấu hình SignalTap:



Hình 24 Cấu hình SignalTap

- Code xử lý theo yêu cầu:

```
#include <stdio.h>
#include "system.h"
#include <unistd.h>

int main()
{
    volatile int *sw_ptr      = (int *) SWITCHES_0_BASE;
    volatile int *compute_ptr = (int *) COMPUTE_0_BASE;
    volatile int *led_ptr     = (int *) DECODER_7_SEGMENT_0_BASE;

    int sw, a, b, x, y;

    while(1)
    {
        sw = *sw_ptr;
        a = sw & 0xF;           // SW[3:0]
        b = (sw >> 4) & 0xF;   // SW[7:4]
        x = (sw >> 8) & 0xF;    // SW[11:8]

        *(compute_ptr + 0) = a;
        *(compute_ptr + 1) = b;
        *(compute_ptr + 2) = x;

        y = *(compute_ptr + 3);

        *led_ptr = y & 0xFF;

        usleep(100000); // 100ms
    }

    return 0;
}
```

Hình 25 Code xử lý yêu cầu

- Video thực hành & kết quả SignalTap (*)

(*) Video nạp KIT thực hành LAB2: [Video](#)